



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623
Database Programming
Semester 1 2025/2026
Assignment 1 (10%)

Lecturer: Shamini A/P Raja Kumaran

Name	:	YASMIN BATRISYIA BINTI ZAHIRUDDIN
Matric Id	:	A23CS0201
Section	:	02

Name	:	NURUL ADRIANA BINTI KAMAL JEFRI
Matric Id	:	A23CS0258
Section	:	02

Task 1 - Database Creation and Integrity Constraints

1. Create a database named hostel_mgmt_yasminana.

```
create database hostel_mgmt_yasminana;  
  
use hostel_mgmt_yasminana;
```

Figure 1.1.1 create database hostel_mgmt_yasminana

2. Create the following tables with entity and referential integrity.

```
create table room_types (  
    type_id int auto_increment primary key,  
    type_name varchar(50) not null unique,  
    rent decimal(10, 2) not null,  
    deposit decimal(10, 2) not null,  
    capacity int not null  
);
```

Figure 1.2.1 create room_types table

```
create table rooms (  
    room_id int auto_increment primary key,  
    room_no varchar(4) not null,  
    floor_no int not null,  
    is_occupied boolean,  
    type_id int,  
    foreign key (type_id) references room_types(type_id)  
);
```

Figure 1.2.2 create rooms table

```
create table students (  
    student_id int auto_increment primary key,  
    fname varchar(50) not null,  
    lname varchar(50) not null,  
    status enum('ACTIVE', 'NON_ACTIVE') not null,  
    checkin_date date not null,  
    email varchar(100) not null,  
    room_id int not null,  
    foreign key (room_id) references rooms(room_id)  
);
```

Figure 1.2.3 create students table

```
create table maintenance(
    maint_id int auto_increment primary key,
    issue_desc varchar(50),
    severity enum('LOW', 'MEDIUM', 'HIGH') not null,
    status enum('OPEN', 'RESOLVED') not null,
    reported_on date not null,
    resolved_on date not null,
    room_id int not null,
    foreign key (room_id) references rooms(room_id)
);
```

Figure 1.2.4 create maintenance table

```
create table payments(
    payment_id int auto_increment primary key,
    amount decimal(10,2) not null,
    paid_on date not null,
    method enum('CASH', 'FPX', 'CARD', 'TNG') not null,
    note varchar(10) not null,
    student_id int,
    foreign key (student_id) references students(student_id)
);
```

	payment_id	amount	paid_on	method	note	student_id
▶	1	350.00	2025-10-26	CASH	october rent	101
	2	450.00	2025-10-28	FPX	october rent	102
	3	550.00	2025-10-25	CARD	october rent	103
	4	350.00	2025-11-05	CASH	november rent	104
	5	1000.00	2025-11-10	CASH	november rent	105
	6	1200.00	2025-10-10	CARD	november rent	106
	7	700.00	2025-10-05	TNG	october deposit	107
	8	1400.00	2025-10-15	FPX	november rent	108
	9	350.00	2025-11-02	CARD	november deposit	109
	10	400.00	2025-11-07	FPX	november deposit	110
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 1.2.5 create payments table

3. Apply constraints: NOT NULL, PRIMARY KEY, FOREIGN KEY, UNIQUE.

```
create table room_types (
    type_id int auto_increment primary key,
    type_name varchar(50) not null unique,
    rent decimal(10, 2) not null,
    deposit decimal(10, 2) not null,
    capacity int not null
);
```

Figure 1.3.1 using NOT NULL, PRIMARY KEY, and UNIQUE in room_types table

```

create table rooms (
    room_id int auto_increment primary key,
    room_no varchar(4) not null,
    floor_no int not null,
    is_occupied boolean,
    type_id int,
    foreign key (type_id) references room_types(type_id)
);

```

Figure 1.3.2 using NOT NULL, PRIMARY KEY, and FOREIGN KEY in rooms table

```

create table students (
    student_id int auto_increment primary key,
    fname varchar(50) not null,
    lname varchar(50) not null,
    status enum('ACTIVE', 'NON_ACTIVE') not null,
    checkin_date date not null,
    email varchar(100) not null,
    room_id int not null,
    foreign key (room_id) references rooms(room_id)
);

```

Figure 1.3.3 using NOT NULL, PRIMARY KEY, and FOREIGN KEY in students table

```

create table maintenance(
    maint_id int auto_increment primary key,
    issue_desc varchar(50),
    severity enum('LOW', 'MEDIUM', 'HIGH') not null,
    status enum('OPEN', 'RESOLVED') not null,
    reported_on date not null,
    resolved_on date not null,
    room_id int not null,
    foreign key (room_id) references rooms(room_id)
);

```

Figure 1.3.4 using NOT NULL, PRIMARY KEY, and FOREIGN KEY in maintenance table

```
create table payments(
    payment_id int auto_increment primary key,
    amount decimal(10,2) not null,
    paid_on date not null,
    method enum('CASH', 'FPX', 'CARD', 'TNG') not null,
    note varchar(10) not null,
    student_id int,
    foreign key (student_id) references students(student_id)
);
```

Figure 1.3.5 using NOT NULL, PRIMARY KEY, and FOREIGN KEY in payments table

4. Create and insert order: room_types → rooms → students → maintenance → payments. A room is 'occupied' if it has ≥ 1 student with status='ACTIVE'; otherwise 'VACANT'. No orphan FKs.
5. Demonstrate schema maintenance:

5.1 ALTER TABLE: add an email column to students and mark it UNIQUE

```
alter table students
modify column email varchar(100) unique;
```

Figure 1.5.5.1 alter students table

5.2 DROP TABLE: create a temporary test table and drop it safely.

```
drop table payments;
```

Figure 1.5.5.2 drop payments table

Task 2 - Data Manipulation and Filtering

1. Data Insertion (DML)

```
insert into room_types (type_name, rent, deposit, capacity)
values ('Single Basic', 350.00, 200.00, 1),
('Single Plus', 450.00, 220.00, 1),
('Double Standard', 550.00, 300.00, 2),
('Double Economy', 350.00, 200.00, 2),
('Premium Queen', 1000.00, 500.00, 2),
('Premium King', 1200.00, 600.00, 2),
('Family Standard', 1400.00, 700.00, 4),
('Family Deluxe', 1600.00, 700.00, 4),
('Studio Compact', 800.00, 350.00, 1),
('Studio Loft', 900.00, 400.00, 2);
```

	type_id	type_name	rent	deposit	capacity
▶	1	Single Basic	350.00	200.00	1
	2	Single Plus	450.00	220.00	1
	3	Double Standard	550.00	300.00	2
	4	Double Economy	350.00	200.00	2
	5	Premium Queen	1000.00	500.00	2
	6	Premium King	1200.00	600.00	2
	7	Family Standard	1400.00	700.00	4
	8	Family Deluxe	1600.00	700.00	4
	9	Studio Compact	800.00	350.00	1
	10	Studio Loft	900.00	400.00	2
*	NULL	NULL	NULL	NULL	NULL

Figure 2.1.1 insert data into room_types table

```

insert into rooms(room_no, floor_no, is_occupied)
values ('A101', 1, TRUE),
('A102', 1, TRUE),
('A103', 1, FALSE),
('A104', 1, FALSE),
('A201', 2, TRUE),
('A202', 2, FALSE),
('A203', 2, FALSE),
('A301', 3, TRUE),
('A302', 3, TRUE),
('A303', 3, FALSE);

```

	room_id	room_no	floor_no	is_occupied	type_id
1		A101	1	1	1
2		A102	1	0	2
3		A103	1	1	3
4		A104	1	1	4
5		A201	2	0	5
6		A202	2	1	6
7		A203	2	0	7
8		A301	3	1	8
9		A302	3	0	9
10		A303	3	1	10
11		A401	4	0	1
12		A402	4	0	2
13		A403	4	0	3
14		A501	5	0	1
15		A502	5	0	1
*	NULL	NULL	NULL	NULL	NULL

Figure 2.1.2 insert data into rooms table

```

SELECT room_id, room_no FROM rooms;

UPDATE rooms
SET type_id = 1
WHERE room_id = 1;

UPDATE rooms
SET type_id = 2
WHERE room_id = 2;

UPDATE rooms
SET type_id = 3
WHERE room_id = 3;

UPDATE rooms
SET type_id = 4
WHERE room_id = 4;

UPDATE rooms
SET type_id = 5
WHERE room_id = 5;

```

```

UPDATE rooms
SET type_id = 6
WHERE room_id = 6;

UPDATE rooms
SET type_id = 7
WHERE room_id = 7;

UPDATE rooms
SET type_id = 8
WHERE room_id = 8;

UPDATE rooms
SET type_id = 9
WHERE room_id = 9;

UPDATE rooms
SET type_id = 10
WHERE room_id = 10;

```

Figure 2.1.3 and Figure 2.1.4 update type_id and room_id of rooms table

```

insert into rooms(room_no, floor_no, is_occupied, type_id)
values ('A401', 4, TRUE, 1),
('A402', 4, TRUE, 2),
('A403', 4, FALSE, 3),
('A501', 5, FALSE, 1),
('A502', 5, TRUE, 1);

```

Figure 2.1.5 insert new data into rooms table

```

insert into students (room_id, fname, lname, status, checkin_date, email)
values (4, 'Adibah', 'Noor', 'ACTIVE', '2023-11-01', 'adibah.noor@gmail.com'),
(1, 'Aryan', 'Nik', 'ACTIVE', '2022-10-21', 'nik_aryan@gmail.com'),
(2, 'Fahmi', 'Nazu', 'NON_ACTIVE', '2024-09-04', 'nazuFahmi@gmail.com'),
(6, 'Kashwi', 'Verangan', 'ACTIVE', '2021-05-31', 'kashwi00@gmail.com'),
(9, 'Harry', 'Whistledown', 'NON_ACTIVE', '2022-04-28', 'sirHarry@gmail.com'),
(3, 'Dhea', 'Nang', 'ACTIVE', '2023-05-23', 'nang_dhea@gmail.com'),
(8, 'Yasmin', 'Hana', 'ACTIVE', '2021-07-10', 'princess.hana@gmail.com'),
(7, 'Asri', 'Faqih', 'NON_ACTIVE', '2024-12-02', '00asri00@gmail.com'),
(5, 'Darwin', 'Khair', 'NON_ACTIVE', '2023-05-22', 'kingDarwin@gmail.com'),
(10, 'James', 'Fahrin', 'ACTIVE', '2021-01-15', 'james_fahrin@gmail.com');

```

	student_id	fname	lname	status	checkin_date	email	room_id
▶	101	Adibah	Noor	ACTIVE	2023-11-01	adibah.noor@gmail.com	4
	102	Aryan	Nik	ACTIVE	2022-10-21	nik_aryan@gmail.com	1
	103	Fahmi	Nazu	NON_ACTIVE	2024-09-04	nazuFahmi@gmail.com	2
	104	Kashwi	Verangan	ACTIVE	2021-05-31	kashwi00@gmail.com	6
	105	Harry	Whistledown	NON_ACTIVE	2022-04-28	sirHarry@gmail.com	9
	106	Dhea	Nang	ACTIVE	2023-05-23	nang_dhea@gmail.com	3
	107	Yasmin	Hana	ACTIVE	2021-07-10	princess.hana@gmail.com	8
	108	Asri	Faqih	NON_ACTIVE	2024-12-02	00asri00@gmail.com	7
	109	Darwin	Khair	NON_ACTIVE	2023-05-22	kingDarwin@gmail.com	5
	110	James	Fahrin	ACTIVE	2021-01-15	james_fahrin@gmail.com	10
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.1.6 insert data into students table

```

update students set student_id = 101 where student_id = 1;
update students set student_id = 102 where student_id = 2;
update students set student_id = 103 where student_id = 3;
update students set student_id = 104 where student_id = 4;
update students set student_id = 105 where student_id = 5;
update students set student_id = 106 where student_id = 6;
update students set student_id = 107 where student_id = 7;
update students set student_id = 108 where student_id = 8;
update students set student_id = 109 where student_id = 9;
update students set student_id = 110 where student_id = 10;

```

Figure 2.1.7 update student_id in students table

```

insert into maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
values (1, 'Water Leak', 'LOW', 'RESOLVED', '2025-09-11', '2025-10-01'),
(2, 'No Electricity', 'HIGH', 'OPEN', '2025-10-12', '2025-10-12'),
(3, 'Flying Roof', 'HIGH', 'RESOLVED', '2025-09-17', '2025-11-09'),
(4, 'Ruptured Wall', 'MEDIUM', 'RESOLVED', '2025-11-01', '2025-11-12'),
(5, 'Burnt Lamp', 'LOW', 'RESOLVED', '2025-10-28', '2025-11-11'),
(6, 'Water Leak', 'HIGH', 'OPEN', '2025-05-12', '2025-06-12'),
(7, 'Flying Roof', 'MEDIUM', 'OPEN', '2025-07-15', '2025-07-15'),
(8, 'Flying Roof', 'HIGH', 'OPEN', '2025-06-12', '2025-06-12'),
(9, 'Ruptured Wall', 'LOW', 'RESOLVED', '2025-09-15', '2025-12-15'),
(10, 'Burnt Lamp', 'MEDIUM', 'RESOLVED', '2025-03-15', '2025-05-15');

```

	maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
▶	25	Water Leak	LOW	RESOLVED	2025-09-11	2025-10-01	1
	26	No Electricity	HIGH	OPEN	2025-10-12	2025-10-12	2
	30	Water Leak	HIGH	OPEN	2025-05-12	2025-06-12	6
	31	Flying Roof	MEDIUM	OPEN	2025-07-15	2025-07-15	7
	32	Flying Roof	HIGH	OPEN	2025-06-12	2025-06-12	8
	34	Burnt Lamp	MEDIUM	RESOLVED	2025-03-15	2025-05-15	10
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.1.8 insert data into maintenance table

```

alter table maintenance modify column resolved_on date null;

```

Figure 2.1.9 alter maintenance table

```

alter table payments modify column note varchar(100);

```

Figure 2.1.10 alter payments table


```

insert into payments(amount, paid_on, method, note, student_id)
values (350.00, '2025-10-26', 'CASH', 'october rent', 101),
(450.00, '2025-10-28', 'FPX', 'october rent', 102),
(550.00, '2025-10-25', 'CARD', 'october rent', 103),
(350.00, '2025-11-05', 'CASH', 'november rent', 104),
(1000.00, '2025-11-10', 'CASH', 'november rent', 105),
(1200.00, '2025-10-10', 'CARD', 'november rent', 106),
(700.00, '2025-10-05', 'TNG', 'october deposit', 107),
(1400.00, '2025-10-15', 'FPX', 'november rent', 108),
(350.00, '2025-11-02', 'CARD', 'november deposit', 109),
(400.00, '2025-11-07', 'FPX', 'november deposit', 110);

```

	payment_id	amount	paid_on	method	note	student_id
▶	1	350.00	2025-10-26	CASH	october rent	101
	2	450.00	2025-10-28	FPX	october rent	102
	3	550.00	2025-10-25	CARD	october rent	103
	4	350.00	2025-11-05	CASH	november rent	104
	5	1000.00	2025-11-10	CASH	november rent	105
	6	1200.00	2025-10-10	CARD	november rent	106
	7	700.00	2025-10-05	TNG	october deposit	107
	8	1400.00	2025-10-15	FPX	november rent	108
	9	350.00	2025-11-02	CARD	november deposit	109
	10	400.00	2025-11-07	FPX	november deposit	110
✱	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.1.11 insert data into payments table

2. UPDATE and DELETE Operations

2.1 UPDATE Operations

```

SET SQL_SAFE_UPDATES = 0;

UPDATE rooms
SET is_occupied = CASE
WHEN room_id IN (
SELECT room_id
FROM students
WHERE status = 'ACTIVE'
) THEN 1
ELSE 0
END;

```

Figure 2.1.1 SQL Command

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	1
2	A102	1	1	2
3	A103	1	0	3
4	A104	1	0	4
5	A201	2	1	5
6	A202	2	0	6
7	A303	3	0	7
8	A301	3	1	8
9	A302	3	1	9
10	A303	3	0	10
11	A401	4	1	1
12	A402	4	1	2
13	A403	4	0	3
14	A501	5	0	1

Figure 2.1.2 Before UPDATE

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	1
2	A102	1	0	2
3	A103	1	1	3
4	A104	1	1	4
5	A201	2	0	5
6	A202	2	1	6
7	A203	2	0	7
8	A301	3	1	8
9	A302	3	0	9
10	A303	3	1	10
11	A401	4	0	1
12	A402	4	0	2
13	A403	4	0	3
14	A404	4	0	4

Figure 2.1.3 After UPDATE

Before this we just randomly insert the data. But, after we update the `is_occupied` column based on the condition, the `is_occupied` column data now is changed and it is more correct.

2.2. DELETE Operations

```
DELETE FROM maintenance
WHERE status = 'RESOLVED'
AND reported_on > (CURDATE() - INTERVAL 60 DAY);
```

Figure 2.2.1 SQL Command DELETE

maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
25	Water Leak	LOW	RESOLVED	2025-09-11	2025-10-01	1
26	No Electricity	HIGH	OPEN	2025-10-12	2025-10-12	2
27	Flying Roof	HIGH	RESOLVED	2025-09-17	2025-11-09	3
28	Ruptured Wall	MEDIUM	RESOLVED	2025-11-01	2025-11-12	4
29	Burnt Lamp	LOW	RESOLVED	2025-10-28	2025-11-11	5
30	Water Leak	HIGH	OPEN	2025-05-12	2025-06-12	6
31	Flying Roof	MEDIUM	OPEN	2025-07-15	2025-07-15	7
32	Flying Roof	HIGH	OPEN	2025-06-12	2025-06-12	8
33	Ruptured Wall	LOW	RESOLVED	2025-09-15	2025-12-15	9
34	Burnt Lamp	MEDIUM	RESOLVED	2025-03-15	2025-05-15	10

Figure 2.2.2 Before DELETE

maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
25	Water Leak	LOW	RESOLVED	2025-09-11	2025-10-01	1
26	No Electricity	HIGH	OPEN	2025-10-12	2025-10-12	2
30	Water Leak	HIGH	OPEN	2025-05-12	2025-06-12	6
31	Flying Roof	MEDIUM	OPEN	2025-07-15	2025-07-15	7
32	Flying Roof	HIGH	OPEN	2025-06-12	2025-06-12	8
34	Burnt Lamp	MEDIUM	RESOLVED	2025-03-15	2025-05-15	10

Figure 2.2.3 After DELETE

We applied the DELETE Operation to delete maintenance records where `status = 'RESOLVED'` and the reported date is older than 60 days. Figure 2.2.3 shows the result after DELETE operations.

3. Data Retrieval and Filtering Queries

3.1. BETWEEN

The screenshot shows a database query editor with the following SQL commands:

```

215 • DELETE FROM maintenance;
216
217 • DELETE FROM maintenance
218   WHERE status = 'RESOLVED'
219   AND reported_on > (CURDATE() - INTERVAL 60 DAY);
220
221 • select * from room_types where rent > 400 and rent < 800;

```

The result grid displays the following data:

type_id	type_name	rent	deposit	capacity
2	Single Plus	450.00	220.00	1
3	Double Standard	550.00	300.00	2

Figure 3.1 Query using BETWEEN

Filtering the data using BETWEEN and shows data column room types where rent is between RM400 and RM800.

3.2. LIKE

The screenshot shows a database query editor with the following SQL commands:

```

216
217 • DELETE FROM maintenance
218   WHERE status = 'RESOLVED'
219   AND reported_on > (CURDATE() - INTERVAL 60 DAY);
220
221 • select * from room_types where rent > 400 and rent < 800;
222
223 • select * from students where fname like 'A%';

```

The result grid displays the following data:

student_id	fname	lname	status	checkin_date	email	room_id
101	Adbah	Noor	ACTIVE	2023-11-01	adibah.noor@gmail.com	4
102	Aryan	Nik	ACTIVE	2022-10-21	nik_arian@gmail.com	1
108	Arsi	Fagih	NON_ACTIVE	2024-12-02	00asr00@gmail.com	7

Figure 3.2 Query using LIKE

Display student name based on their first name that are starting with letter A as we used 'A%'

3.3 IN

The screenshot shows a database query editor with the following SQL commands:

```

218   WHERE status = 'RESOLVED'
219   AND reported_on > (CURDATE() - INTERVAL 60 DAY);
220
221 • select * from room_types where rent > 400 and rent < 800;
222
223 • select * from students where fname like 'A%';
224
225 • select * from payments where method in ('FPX', 'CARD');

```

The result grid displays the following data:

payment_id	amount	paid_on	method	note	student_id
2	450.00	2025-10-28	FPX	october rent	102
3	550.00	2025-10-25	CARD	october rent	103
6	1200.00	2025-10-10	CARD	november rent	106
8	1400.00	2025-10-15	FPX	november rent	108
9	350.00	2025-11-02	CARD	november deposit	109
10	400.00	2025-11-07	FPX	november deposit	110

Figure 3.3 Query using IN

Display payments data where payment method in FPX or CARD.

3.4 Combining AND, OR and NOT

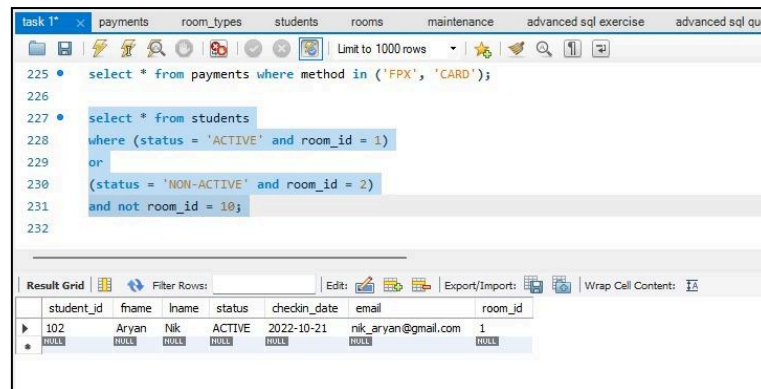


Figure 3.4 Combining query

Select students where (status is active AND room_id is 1) OR (status is non-active AND room_id is 2) AND the room_id is NOT 10.

4. Functions and Expressions

4.1 Aggregate Function

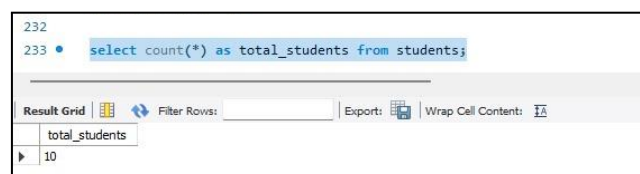


Figure 4.1.1 COUNT function

Figure 4.1.1 display how many total students from the students table

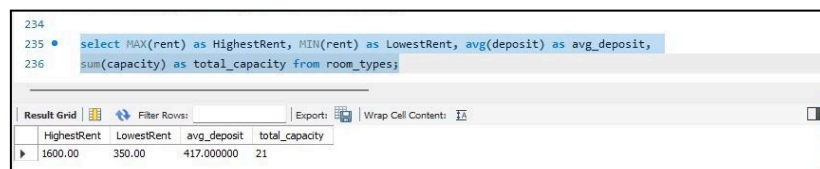


Figure 4.1.2 Combination aggregate function

Figure 4.1.2 display the highest rent by using MAX, lowest rent by using MIN, average deposit by using AVG, and total capacity for all room types by using SUM from the table room_types.

4.2 String Functions

237

238 • `select fName, UPPER(fName) as UpperFirstName from students;`

Result Grid

fName	UpperFirstName
Adbah	ADIBAH
Aryan	ARYAN
Fahmi	FAHMI
Kashwi	KASHWI
Harry	HARRY
Dhea	DHEA
Yasmin	YASMIN
Asri	ASRI
Darwin	DARWIN
James	JAMES

Figure 4.2.1 UPPER function

Figure 4.2.1 displays how the function UPPER works by changing the previous last name where only the first letter is upper to the whole name is using uppercase.

243

244 • `select fName, lName,`

245 `CONCAT(fName, ' ', lName) as FullName from students;`

Result Grid

fName	lName	FullName
Adbah	Noor	Adbah Noor
Aryan	Nik	Aryan Nik
Fahmi	Nazu	Fahmi Nazu
Kashwi	Verangan	Kashwi Verangan
Harry	Whistledown	Harry Whistledown
Dhea	Nang	Dhea Nang
Yasmin	Hana	Yasmin Hana
Asri	Faqh	Asri Faqh
Darwin	Khair	Darwin Khair
James	Fahrin	James Fahrin

Figure 4.2.2 CONCAT function

Figure 4.2.2 displays the combination of first name and last name under one column name Full Name.

Task 3 - Apply SQL functions in report queries

1. Create a view

```
create view v_room_status as
select
    r.room_no,
    rt.type_name,
    rt.rent,
    r.floor_no,
    rt.capacity,
    sum(case when s.status = 'ACTIVE' then 1 else 0 end) as n_occupants,
    sum(case when m.status = 'OPEN' then 1 else 0 end) as pending_issues,
    case
        when sum(case when s.status = 'ACTIVE' then 1 else 0 end) = 0
        then TRUE
        else FALSE
    end as is_vacant
from rooms r
left join room_types rt
    on r.type_id = rt.type_id
left join students s
    on s.room_id = r.room_id
left join maintenance m
    on m.room_id = r.room_id
group by r.room_no,
    rt.type_name,
    rt.rent,
    r.floor_no,
    rt.capacity;
```

Figure 3.1.1 Creating view SQL

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Single Basic	350.00	1	1	1	0	0
A102	Single Plus	450.00	1	1	0	1	1
A103	Double Standard	550.00	1	2	1	0	0
A104	Double Economy	350.00	1	2	1	0	0
A201	Premium Queen	1000.00	2	2	0	0	1
A202	Premium King	1200.00	2	2	1	1	0
A203	Family Standard	1400.00	2	4	0	1	1
A301	Family Deluxe	1600.00	3	4	1	1	0
A302	Studio Compact	800.00	3	1	0	0	1
A303	Studio Loft	900.00	3	2	1	0	0
A401	Single Basic	350.00	4	1	0	0	1
A402	Single Plus	450.00	4	1	0	0	1
A403	Double Standard	550.00	4	2	0	0	1
A501	Single Basic	350.00	5	1	0	0	1
A502	Single Basic	350.00	5	1	0	0	1

Figure 3.1.2 View table

Created a view v_room_status that include the columns room_no, type_name, rent, floor_no, capacity, n_occupants, pending_issues, and is_vacant, where n_occupants represents the number of students in each room with a status of 'ACTIVE', pending_issues represents the number of maintenance records with a status of 'OPEN' for each room, and is_vacant is set to TRUE(1) when n_occupants equals to zero.

2. Summary queries

2.1 Total number of students per room type

```

274 • select
275     rt.type_name,
276     count(s.student_id) as total_student
277 from room_types rt
278 left join rooms r
279     on r.type_id = rt.type_id
280 left join students s

```

type_name	total_student
Double Economy	1
Double Standard	1
Family Deluxe	1
Family Standard	1
Premium King	1
Premium Queen	1
Single Basic	1
Single Plus	1
Studio Compact	1
Studio Loft	1

Figure 2.1 COUNT function

Display the total students for each room type.

2.2 Average rent and total deposit per room type

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```

282 group by rt.type_name;
283
284 select
285     round(avg(rent),2) as Average_rent,
286     round(sum(deposit),2) as total_deposit
287 from room_types;
288
289

```

The result grid displays the following data:

Average_rent	total_deposit
860.00	4170.00

Figure 2.2 Average function with 2 decimal places

Display the average rent and total deposit with values rounded to 2 decimal places.

2.3 Monthly payment totals grouped by year & month

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```

288
289 select
290     year(paid_on) as year,
291     month(paid_on) as month,
292     sum(amount) as total_amount
293 from payments
294 group by year(paid_on), month(paid_on)
295 order by year, month;
296
297

```

The result grid displays the following data:

year	month	total_amount
2025	10	4650.00
2025	11	2100.00

Figure 2.3 Group by function

Display the Monthly payment totals grouped by year & month

2.4 Count of OPEN maintenance issues per floor using HAVING COUNT > 2

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```

296
297 select
298     count(case when m.status='OPEN' then 1 end) as num_issues,
299     r.floor_no
300 from maintenance m
301 left join rooms r
302     on r.room_id = m.room_id
303 group by r.floor_no
304 having count(case when m.status='OPEN' then 1 end) > 2;
305

```

The result grid displays the following data:

num_issues	floor_no
------------	----------

Figure 2.4 Count of OPEN maintenance

Based on our data, there are no issues that is more than 2 for each floor.

3. Apply SQL functions in our report queries; ROUND(), UPPER()/LOWER(), CONCAT(), and CASE to categorize room type rent as LOW / MEDIUM / HIGH (state your thresholds explicitly)

```
SELECT
    r.room_no,
    ROUND(rt.rent, 2) AS rent_rounded,
    CONCAT('Room ', r.room_no, ' - ', rt.type_name) AS room_label,
    CASE
        WHEN rt.rent <= 100 THEN 'LOW'
        WHEN rt.rent <= 200 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS rent_category
FROM rooms r
JOIN room_types rt
ON r.type_id = rt.type_id;
```

Figure 3.1 Report Queries SQL

room_no	type_name	rounded_rent	type_upper	location	rent_category	occupancy_status
A101	Single Basic	350.00	SINGLE BASIC	Floor 1 - A101	LOW	OCCUPIED
A102	Single Plus	450.00	SINGLE PLUS	Floor 1 - A102	MEDIUM	VACANT
A103	Double Standard	550.00	DOUBLE STANDARD	Floor 1 - A103	MEDIUM	OCCUPIED
A104	Double Economy	350.00	DOUBLE ECONOMY	Floor 1 - A104	LOW	OCCUPIED
A201	Premium Queen	1000.00	PREMIUM QUEEN	Floor 2 - A201	HIGH	VACANT
A202	Premium King	1200.00	PREMIUM KING	Floor 2 - A202	HIGH	OCCUPIED
A203	Family Standard	1400.00	FAMILY STANDARD	Floor 2 - A203	HIGH	VACANT
A301	Family Deluxe	1600.00	FAMILY DELUXE	Floor 3 - A301	HIGH	OCCUPIED
A302	Studio Compact	800.00	STUDIO COMPACT	Floor 3 - A302	MEDIUM	VACANT
A303	Studio Loft	900.00	STUDIO LOFT	Floor 3 - A303	HIGH	OCCUPIED
A401	Single Basic	350.00	SINGLE BASIC	Floor 4 - A401	LOW	VACANT
A402	Single Plus	450.00	SINGLE PLUS	Floor 4 - A402	MEDIUM	VACANT
A403	Double Standard	550.00	DOUBLE STANDARD	Floor 4 - A403	MEDIUM	VACANT
A501	Single Basic	350.00	SINGLE BASIC	Floor 5 - A501	LOW	VACANT
A502	Single Basic	350.00	SINGLE BASIC	Floor 5 - A502	LOW	VACANT

Figure 3.2 Report Queries

Display report where we uppercase the room type name, rounded the rent into 2 decimal places, combining the floor with room no for easier location and indicating the occupancy status.

<pre>SELECT 'room_types' AS table_name, COUNT(*) AS record_count FROM room_types UNION ALL SELECT 'rooms' AS table_name, COUNT(*) AS record_count FROM rooms UNION ALL</pre>	<pre>SELECT 'maintenance' AS table_name, COUNT(*) AS record_count FROM maintenance UNION ALL SELECT 'payments' AS table_name, COUNT(*) AS record_count FROM payments;</pre>
--	---

Figure 3.3 short summary table listing all created tables and the number of records per table

	table_name	record_count
▶	room_types	10
	rooms	15
	students	10
	maintenance	6
	payments	10

Figure 3.3 short summary table

This summary shows that all five required tables have been created with at least 10 records each, meeting the assignment requirements. The *rooms* table has 15 records, which exceeds the minimum requirement of 10 records per table.

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more).	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			